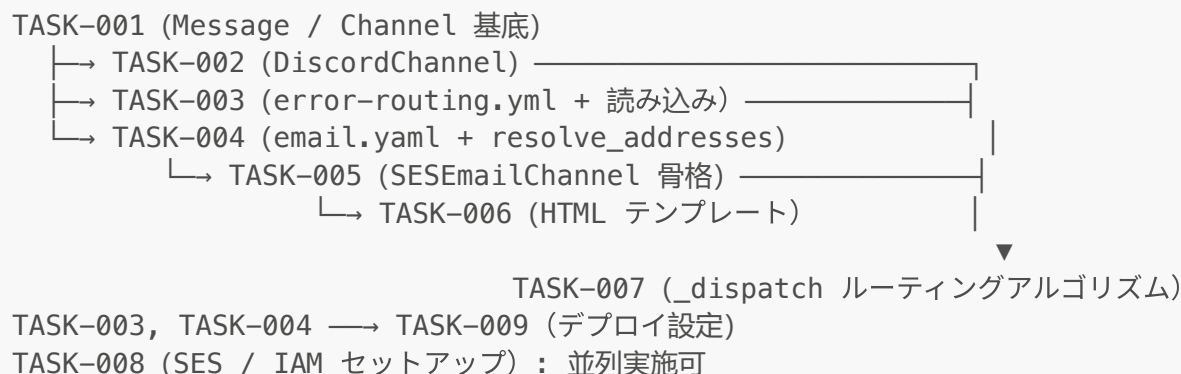


S3データ不着検知時のクライアントへのメール通知 — 実装タスク

- **SPEC:** client-email-notification@0.4.0
- **rev:** 3

実装順序（依存関係）



タスク一覧

TASK-001: Message dataclass と Channel 基底クラスの実装

- **REQ:** REQ-002
- **依存:** なし
- **完了基準:** `src/channels/message.py` と `src/channels/base.py` が存在し、mypy エラーがない

`Message` (frozen dataclass) と `Channel` (ABC) を新設する。 `send()` の引数型は `Message` とし、Discord にも SES にも依存しない。

実装ガイド:

```

# src/channels/message.py
@dataclass(frozen=True)
class Message:
    title: str
    severity: str          # HIGH / MEDIUM / LOW
    confidence: str        # high / medium / low
    root_cause: str
    actions: list[str]
    alarm_name: str
    ship_name: str
    timestamp: datetime

# src/channels/base.py
class Channel(ABC):
    @property
  
```

```
@abstractmethod
def id(self) -> str: ...

@abstractmethod
def send(self, message: Message) -> None: ...
```

TASK-002: DiscordChannel の実装（既存コードの移植）

- **REQ:** REQ-002
- **依存:** TASK-001
- **完了基準:** `DiscordChannel.send(message)` が既存と同等の Discord embed を送信する。main.py から discord_webhook を直接呼ぶコードが削除される

main.py 内の Discord 送信ロジックを `src/channels/discord.py` に移植する。`Message` → `DiscordEmbed` の変換は `_to_embed()` 内部で行い、discord_webhook ライブラリへの依存をこのクラスに閉じ込める。

main.py から削除するコード

対象	現在の場所	対応
<code>from discord_webhook import DiscordEmbed, DiscordWebhook</code>	line 35	discord.py 内に移動
<code>DISCORD_SEVERITY_COLOR</code> 定数	lines 66-73	discord.py 内の定数として移動
<code>main()</code> 内の Discord embed 送信ブロック	lines 723-769	TASK-007 で <code>_dispatch()</code> 呼び出しに置き換え

main.py に残すコード（Channel 抽象化の対象外）

- `_post_minimal_embed()`: alarm 名不正・log group 未設定・AssumeRole 失敗・Bedrock 失敗時の Discord 専用フォールバック。クライアント通知ではないため Channel を経由しない
- `_post_prompt_attachment()`: Bedrock prompt を Discord に添付するデバッグ機能。Channel とは無関係

実装ガイド:

```
# src/channels/discord.py
DISCORD_SEVERITY_COLOR: dict[str, int] = {
    "LOW": 0x2ECC71,
    "MEDIUM": 0xF1C40F,
    "HIGH": 0xE74C3C,
}

class DiscordChannel(Channel):
    def __init__(self, webhook_url: str) -> None:
```

```

        self._webhook_url = webhook_url

    @property
    def id(self) -> str:
        return "discord"

    def send(self, message: Message) -> None:
        embed = self._to_embed(message)
        webhook = DiscordWebhook(url=self._webhook_url)
        webhook.add_embed(embed)
        webhook.execute()

    def _to_embed(self, message: Message) -> DiscordEmbed:
        # severity -> color マッピング (既存定数を流用)
        # title / fields / footer を Message から組み立てる
        # ※ 件数・集計時間窓・deeplinks は Message に含まれないため embed に出さな

```

TASK-003: error-routing.yml の作成と読み込み実装

- **REQ:** REQ-005
- **依存:** なし
- **完了基準:** config/error-routing.yml が存在し、_load_error_routing() が {error_id: entry} の辞書を返す。_resolve_channel_ids() は error_id 不在時に ["discord"] を返し WARNING ログを出す。_resolve_error_id() は log_rows の有無で s3_data_missing / lambda_failure を返す

実装ガイド:

```

# config/error-routing.yml
- id: s3_data_missing
  channels:
    - discord
    - email.dev
    - email.sakura
  description: S3へのデータ不着を検知したとき (Lambda未起動)

- id: lambda_failure
  channels:
    - discord
    - email.dev
    - email.sakura
  description: Lambdaが起動したが処理中に失敗したとき

```

```

def _resolve_error_id(alarm_name: str, log_rows: list) -> str:
    if not ALARM_NAME_RE.match(alarm_name):
        logger.warning("unknown alarm_name pattern %r", alarm_name)
        return "unknown"

```

```
if not log_rows:
    return "s3_data_missing"
return "lambda_failure"

def _load_error_routing() -> dict[str, dict]:
    path = Path(__file__).parent / "config" / "error-routing.yml"
    entries: list[dict] = yaml.safe_load(path.read_text(encoding="utf-8"))
    return {e["id"]: e for e in entries}

def _resolve_channel_ids(error_id: str, routing: dict[str, dict]) -> list[str]:
    entry = routing.get(error_id)
    if entry is None:
        logger.warning("error_id %r not found in error-routing.yml, falling back to discord", error_id)
        return ["discord"]
    return entry["channels"]
```

TASK-004: email.yaml の作成と resolve_addresses 実装

- **REQ:** REQ-006
- **依存:** なし
- **完了基準:** config/email.yaml が存在し、resolve_addresses(group_id, groups) が group_id に対応するアドレスリストを返す。group_id 不在の場合は [] + WARNING

実装ガイド:

```
# config/email.yaml - リスト of dict、id でグループ識別、add に送信先アドレス
- id: dev
  add:
    - alerts@scrumsign.com

- id: sakura
  add:
    - captain@sakura-shipping.com
    - ops@sakura-shipping.com
```

```
def resolve_addresses(group_id: str, entries: list[dict]) -> list[str]:
    by_id = {e["id"]: e for e in entries}
    entry = by_id.get(group_id)
    if entry is None:
        logger.warning("email group %r not found in email.yaml", group_id)
        return []
    return list(entry.get("add", []))
```

TASK-005: SESEmailChannel の骨格実装

- **REQ:** REQ-001, REQ-002
- **依存:** TASK-001, TASK-004
- **完了基準:** `SESEmailChannel.send(message)` が SES `send_email` API を呼ぶ。アドレスリストが空の場合は送信スキップ + WARNING ログ。 `id` プロパティが `"email.<group_id>"` を返す

HTML テンプレートの中身は TASK-006 で実装する。本タスクでは SES 呼び出し骨格のみ。

実装ガイド:

```
class SESEmailChannel(Channel):
    def __init__(self, group_id: str) -> None:
        self._group_id = group_id
        entries: list[dict] = yaml.safe_load(
            (Path(__file__).parent.parent / "config" /
             "email.yaml").read_text()
        )
        self._addresses = resolve_addresses(group_id, entries)

    @property
    def id(self) -> str:
        return f"email.{self._group_id}"

    def send(self, message: Message) -> None:
        if not self._addresses:
            logger.warning("no email addresses for group %r, skipping",
                           self._group_id)
            return
        boto3.client("ses").send_email(
            Source=os.environ["SES_FROM_ADDRESS"],
            Destination={"ToAddresses": self._addresses},
            Message={
                "Subject": {"Data": message.title, "Charset": "UTF-8"},
                "Body": {
                    "Html": {"Data": self._to_html(message), "Charset":
"UTF-8"},
                    "Text": {"Data": self._to_plain(message), "Charset":
"UTF-8"},
                },
            },
        )
```

TASK-006: HTML テンプレートの実装

- **REQ:** REQ-003, REQ-004
- **依存:** TASK-005
- **完了基準:** `_to_html(message)` の返り値に `ship_name` ・ JST タイムスタンプ ・ `severity` ・ `actions` が含まれる。 `_to_plain(message)` も同内容のプレーンテキストを返す

実装ガイド:

```

def _to_html(self, message: Message) -> str:
    jst = message.timestamp.astimezone(ZoneInfo("Asia/Tokyo"))
    actions_html = "".join(f"<li>{a}</li>" for a in message.actions)
    return (
        f"<h2>[{message.severity}] {message.title}</h2>"
        f"<p><b>対象</b>: {message.ship_name}</p>"
        f"<p><b>検知時刻</b>: {jst:%Y-%m-%d %H:%M JST}</p>"
        f"<p><b>原因推定</b>: {message.root_cause}</p>"
        f"<ul>{actions_html}</ul>"
    )

def _to_plain(self, message: Message) -> str:
    jst = message.timestamp.astimezone(ZoneInfo("Asia/Tokyo"))
    actions = "\n".join(f"- {a}" for a in message.actions)
    return (
        f"[{message.severity}] {message.title}\n"
        f"対象: {message.ship_name}\n"
        f"検知時刻: {jst:%Y-%m-%d %H:%M JST}\n"
        f"原因推定: {message.root_cause}\n"
        f"推奨アクション:\n{actions}"
    )

```

TASK-007: ルーティングアルゴリズム (`_dispatch`) の実装

- **REQ:** REQ-007
- **依存:** TASK-002, TASK-003, TASK-005
- **完了基準:** `_dispatch(alarm_name, message, error_id)` が3段階ルーティングを通じて正しいチャンネルのみに送信する。各フォールバック条件で WARNING ログが出る。一方の Channel が例外を出しても他方が継続される。ログ有無にかかわらず Bedrock と `_dispatch` が呼ばれる

main.py の既存 Discord embed 送信ブロック (lines 723–769) と 0 件ログの早期 return (lines 649–662) の両方を、`_resolve_error_id` → Bedrock → `Message` 構築 → `_dispatch()` の共通フローに置き換える。DESIGN.md の「ルーティングアルゴリズム詳細」を参照して実装する。

main.py への追加関数

`_load_error_routing` (TASK-003) , `_resolve_channel_ids` (TASK-003) を main.py に追加済みの前提で、以下を追加する:

```

from src.channels.message import Message
from src.channels.discord import DiscordChannel
from src.channels.email import SESEmailChannel

def _resolve_error_id(alarm_name: str, log_rows: list) -> str:
    if not ALARM_NAME_RE.match(alarm_name):
        logger.warning("unknown alarm_name pattern %r", alarm_name)
        return "unknown"
    if not log_rows:
        return "s3_data_missing"

```

```

        return "lambda_failure"

def _build_channel_registry(channel_ids: list[str]) -> dict[str, Channel]:
    registry: dict[str, Channel] = {
        "discord":
DiscordChannel(webhook_url=os.environ["DISCORD_WEBHOOK_URL"]),
    }
    for cid in channel_ids:
        if cid.startswith("email."):
            group_id = cid.split(".", 1)[1]
            registry[cid] = SESEmailChannel(group_id=group_id)
    return registry

def _dispatch(alarm_name: str, message: Message, error_id: str) -> None:
    routing      = _load_error_routing()
    channel_ids  = _resolve_channel_ids(error_id, routing)
    registry     = _build_channel_registry(channel_ids)

    for cid in channel_ids:
        channel = registry.get(cid)
        if channel is None:
            logger.warning("channel_id %r not in registry, skipping", cid)
            continue
        try:
            channel.send(message)
        except Exception:
            logger.warning("channel %r send failed", cid, exc_info=True)
            # 再 raise しない (AC-007-5)

```

main.py の置き換え箇所

削除: 0 件ログの早期 return ブロック（現 lines 649–662）を削除する。

置き換え: 現在の Discord embed 送信ブロック（lines 723–769）を以下に置き換える:

```

# --- error_id を確定 (ログ有無で s3_data_missing / lambda_failure を判定) ---
error_id = _resolve_error_id(alarm_name, log_rows)

# --- Bedrock で LLM 分析 (log_rows が 0 件の場合も呼び出す) ---
# (既存の Bedrock 呼び出しブロックをここに維持)

# --- Message を構築して全チャンネルにディスパッチ ---
message = Message(
    title=report["summary"],
    severity=report["severity"],
    confidence=report["confidence"],
    root_cause=report["root_cause_hypothesis"],
    actions=report["suggested_actions"],
    alarm_name=alarm_name,
    ship_name=ship_name,
    timestamp=center,
)

```

```

_dispatch(alarm_name, message, error_id)
logger.info("dispatch completed")

return {
    "ok": True,
    "alarm": alarm_name,
    "severity": report.get("severity", ""),
}

```

report → Message フィールドマッピング

Message フィールド	変換元	備考
title	report["summary"]	<code>_normalize_report()</code> で 256 字に切り詰め済み
severity	report["severity"]	HIGH / MEDIUM / LOW
confidence	report["confidence"]	high / medium / low
root_cause	report["root_cause_hypothesis"]	
actions	report["suggested_actions"]	list[str]、最大 3 件
alarm_name	alarm_name	SNS Message から取得済み
ship_name	ship_name	<code>_extract_ship_name_from_alarm_name()</code> の結果
timestamp	center	Alarm 発火時刻の datetime (UTC aware)

TASK-008: IAM ポリシーと SES セットアップ（インフラ）

- **REQ:** REQ-001
- **依存:** なし（並列実施可）
- **完了基準:** Lambda 実行ロールに `ses:SendEmail` / `ses:SendRawEmail` が付与されている。SES サンドボックスが解除されており任意アドレスへの送信が可能

以下を実施する（AWS コンソール / CLI 作業）：

1. SES コンソールで `scrumsign.com` ドメインを登録・DKIM CNAME レコード（3件）を DNS に追加
2. AWS Support ヘサンドボックス解除申請を送信
3. Lambda 実行ロールに IAM インラインポリシーを追加:

```

{
    "Effect": "Allow",
    "Action": ["ses:SendEmail", "ses:SendRawEmail"],
    "Resource": "arn:aws:ses:<region>:<account-id>:identity/scrumsign.com"
}

```


4. `SES_FROM_ADDRESS` を Lambda 環境変数に追加 (例: `alerts@scrumsign.com`)

TASK-009: config ファイルのデプロイ設定確認

- **REQ:** REQ-005, REQ-006
- **依存:** TASK-003, TASK-004
- **完了基準:** `config/error-routing.yml` と `config/email.yml` が Lambda コンテナイメージに含まれる

`config/` ディレクトリは Dockerfile の既存 `COPY config/ ${LAMBDA_TASK_ROOT}/config/` で自動的にイメージに含まれるため、ファイルを配置するだけでよい。Docker ビルド後に `docker run --rm <image> ls /var/task/config/` で両ファイルの存在を確認する。